SPEC-STEERED DEVELOPMENT

A pragmatic take on Spec-Driven Development

With vibe coding, you drift towards one of these:

OPTION A

Read every diff.

Review each change the model makes, line by line, before accepting it. You'll catch the wrong turns and keep quality high — but you're back to writing code at human speed, and a diff without the intent behind it still hides subtle architectural drift.

YOU LOSE THE SPEED.

VS

OPTION B

YOLO it.

Accept the model's changes without reading them. Trust that tests and vibes will catch the mistakes. Move on to the next prompt. You ship fast for a while — until the bugs, drift, and half-understood decisions compound into a codebase no one can reason about.

YOU LOSE THE QUALITY.

Option C: make intent durable.

Vibe coding keeps intent in prompts, chat history, and short-term memory.

SDD moves intent into artifacts the model can reuse: requirements, behaviors, contracts, and acceptance criteria.

THREE FLAVORS OF SDD

01 **Spec-first**
Write the spec before the implementation.

02 **Spec-anchored**
Keep the spec alive as the system evolves.

03 **Spec-as-source**
Treat the spec as canon; regenerate code from it.

WHAT IS SPEC-STEERED DEVELOPMENT?

A lightweight, code-tolerant adaptation of SDD.

The spec remains canon — but discovery does not always start in the spec.

Sometimes you need to try the behavior first: run the app, feel the UX, patch the bug.

START HERE

Code-first experimentation.

Skip the spec when you need to feel the behavior. Prototype, patch, probe — off-spec on purpose.

→

THEN CANONIZE

Promote the learning into the spec.

When the result proves useful, the learning graduates back into canon. The spec stays the source of truth.

THREE ARTIFACTS, STAYING IN SYNC

STABLE

01 · YOU + MODEL, ITERATED

Spec

What must be true.
The stable contract.

ITERATIVE

02 · MODEL DRAFTS, YOU REVIEW

Plan

How we get there, phase by phase.

e.g. Phase 1: hardcoded auth → Phase 3: real OAuth (stays out of spec until it's real)

REGENERATABLE

03 · AGENT TEAM SHIPS

Code

The build output.
Regenerated from spec + plan.

Spec is canonical. Plan is iterative. Code is regeneratable.

THE REPO

```
spec/
  _constitution.md                # project-wide pillars, non-goals
  tracker/
    todo-model.spec.md           # intent + EARS + Given/When/Then
    todo-model.api.v1.yaml
    todo-model.db.v1.yaml
    persistence.spec.md
    persistence.db.v1.yaml
  payments/
    checkout.spec.md
    checkout.api.v1.yaml
    checkout.api.v2.yaml         # breaking change — plan bridges v1 → v2
plan/
  status.md
  phase-1-mvp.md                 # done
  phase-2-checkout-v2.md         # in progress — migrate v1 → v2
  phase-3-offline.md             # draft
code/                             # regenerated from spec + plan
```

THE HABIT THAT MAKES IT WORK

The inner loop.
Implement, review, fix, canonize.

01

Implement phase

Agent team picks up the next phase from the plan and ships the code for it end-to-end.

→

02

Review the result

Run the app and see how it feels. Find where reality diverged from the spec — rough UX, missed cases, things that just don't work. Spot-check the code if something smells off.

→

03

Ad-hoc fix

Ask the agent team to fix what you found — directly, without touching the spec. Fast and targeted, off-spec on purpose.

→

04

Canonize

Promote the ad-hoc fix from code into spec. The tool reads the commit and proposes the spec edit — you review and accept.

REPEAT FOR EVERY PHASE

CANONIZE

The ad-hoc fix graduates into the spec.

THE PRINCIPLE

Every off-spec fix is a signal. The code just learned something the spec didn't know.

Canonizing is the thirty-second habit that turns that signal into durable intent: commit the fix, let the tool read the commit, update the spec.

Skip it, and the same bug comes back with the next regeneration.

THE MECHANISM

01 Patch `code/` — get the app working

02 Commit with intent in the message

03 Tool reads the commit → proposes spec edit

04 You review the diff in `spec/`

05 Next regeneration builds on the new canon

“Implementing the code helps us improve our spec.” — DREW BREUNIG

THE LOOP IN ACTION

Bug: “Completed todos reappear after a page refresh”

Patch code/ first — get the app working again.

Then commit. The canonize tool reads the git commit and updates the spec automatically — no manual edits, no forgetting.

SPEC/003-PERSISTENCE.MD

```
## Requirements

- WHEN a user toggles a todo's completion state,
  THE SYSTEM SHALL persist the new state
  before the next page load.

## Behaviors

Scenario: Completed todo survives refresh
  Given a todo list with one open item
  When  the user marks the item as done
  And   the user refreshes the page
  Then  the item is still shown as done
```

TIPS FROM THE TRENCHES

Habits that pay off fast.

01

Use an agent team, not a single agent.

Split roles — planner, implementer, reviewer. They catch each other's mistakes, and no single context window has to carry the whole project.

02

Delete the code periodically. Keep the spec.

If regenerating produces something worse, the spec has gaps — you found out cheaply. If it produces something better, you just got a free refactor.

03

Canonize every ad-hoc fix.

Thirty seconds in the code, thirty seconds in the spec. Skip the second step and the bug will come back with the next regeneration.

HONEST CAVEATS

Skip SDD when the spec is the code.

01

Throwaway scripts

A 30-line migration you'll run once. Writing a spec is the longer path.

02

Pure exploration

You don't know what you want yet. Vibe first, specify once you've seen it work.

03

Tight, known primitives

A regex, a SQL query, a one-liner. The code already is the spec.

Spec-Steered Development.

GENERATE · REVIEW · AD-HOC FIX · CANONIZE · REPEAT